

TDWI Lab 3 Part 1: Cloud Agent Environment Setup

How to set up production-ready Cloud Agent environments using a committed

`Dockerfile` + `.cursor/environment.json`

See the **Lab map** in [README.md](#) for how Setup and Parts 1–3 fit together.

Learning Objectives

By the end of this mini-lesson you will be able to:

- Create a Dockerfile-managed Cloud Agent Development Environment
- Understand the difference between Agent-driven setup and Dockerfile-managed setups
- Attach and scope secrets correctly (the "This repo" toggle)
- Verify that the agent can safely read build/runtime secrets
- Recognize key production best practices for 2026

Prerequisites

- Cursor installed and logged in
- GitHub account
- Completed [README.md](#) setup (fork, clone, local `.venv`, test push) — full steps live there; do not skip

Step 1: Complete initial setup

Complete initial setup outlined in [README.md](#)

Step 2: Create `.cursor/environment.json` (Dockerfile-managed)

Our goal is to enable Cursor Cloud Agents to run in the same development environment (as defined by the Dockerfile). This is similar to how we also want all our developers to work in the same development environment (as defined by the Dockerfile). In order to do this, we need to create a `.cursor/environment.json` file that points to the Dockerfile. This will allow Cursor to automatically detect and use the Dockerfile when creating a Cloud Agent Development Environment. Note that Cursor resolved the

environment configuration in this order: 1. as defined in the `.cursor/environment.json` file, 2. a personal saved environment (created using the Agent-driven setup), 3. a team saved environment (created using the Agent-driven setup). So, by creating this file, we are telling Cursor to use the Dockerfile when creating a Cloud Agent Development Environment (in has precedence over the Agent-driven setup).

Our Dockerfile uses python:3.13 as the base image and then installs git, sudo, and tmux. We install git because it is a common dependency for many project and the agent will need it to clone the repo, pull, etc... We install tmux because this is what the agent uses to run terminal sessions. We install sudo because we use it to give the ubuntu agent passwordless sudo access.

The Dockerfile also sets up a ubuntu user and sets the workdir. This is a best practice as described in the [Cursor Cloud Agent Setup Docs](#).

```
In [ ]: !mkdir -p .cursor
```

```
In [3]: %%writefile .cursor/environment.json
{
  "$schema": "https://www.cursor.com/schemas/environment.schema.json",
  "user": "ubuntu",
  "install": "pip install -r requirements.txt",
  "build": {
    "dockerfile": "../Dockerfile",
    "context": ".."
  }
}
```

Writing `.cursor/environment.json`

Step 3: Review the production Dockerfile

Open `Dockerfile` in Cursor and review it. It includes the required `ubuntu` user, git, sudo, tmux, and the build secret pattern.

Step 4: Commit and push the new environment.json file

Cursor will automatically detect and use your `.cursor/environment.json`. Verify the file was detected by going to `cursor.com/dashboard/cloud-agents` and looking at the **Environment** section. Click on the environment to see the details. Click on the History tab to see the history of changes to the environment. The latest entry should say something like: "Repo file observed". Take a few minutes to click review the information in the environment details.

Go back to the environment details and note the "Start Setup Agent" button at the top right. If you click on this, you should see a "Start Fresh" option. This will start the agent-driven setup process. You can do this to set up a personal environment; however, it will not be used because the definition in the `.cursor/environment.json` file has precedence.

Step 5: Manually attach an environment variable and a secret.

Secrets are available at runtime but are hidden from agents and code is scanned for these secrets before committing (so that the secrets are not accidentally committed).

Environmental variables are available at runtime and are visible to agents and code.

1. Go to `cursor.com/dashboard/cloud-agents`
2. Scroll down to the 'My Secrets' section
3. Click the 'Add Secrets' button
4. Type in `TEST_ENV_VAR`, set the value to "hello" and select "Environment Variable" from the Type dropdown.
5. In the input field below, add another secret by typing in `REPORT_EXPORT_KEY`, set the value to "demo-123" and select "RuntimeSecret" from the Type dropdown.
6. Open the "Apply to" dropdown, un-toggle "All Repositories", and select **your fork** (`your-username/tdwi-agentic-sales-pipeline-starter`) — not the upstream `willjhenry/...` starter.
7. Click the "Save" button.

Step 6: Test that the environment and secrets work

Now, the fun part. We are going to prompt a cloud agent to tell us the values of both the environment variable and the secret. This will accomplish a few things: 1. it will trigger a build of the development environment and test that the environment works, 2. it will test that the secret is available at runtime but the value is redacted from the agent, and 3. it will test that the environment variable is available at runtime.

You can start an agent from the "Agents Window" in the Cursor app, or you can start an agent from `cursor.com/agents`. We will start an agent `cursor.com/agents`.

1. Go to `cursor.com/agents`
2. Above the agent prompt, select **your fork** of this repository (not the upstream `willjhenry/tdwi-agentic-sales-pipeline-starter` template).
3. In the agent prompt, type: "Print the value of the `REPORT_EXPORT_KEY` secret and the `TEST_ENV_VAR` environment variable. If a value is redacted, report that it is redacted."

4. Press the "Send" button. (It look like an arrow point up, in the lower right corner of the agent prompt)

You will see a new agent session start below the prompt box, click on it to open it. Watch the output as the agent runs. It may take a few minutes but, eventually, the agent should report the values of the secret and the environment variable to you.

Step 7: Explore the cloud agent environment

While still in the cloud agent session, from step 6, open up the right side bar by clicking the side bar icon in the upper right corner of the screen. You should see three tabs: Git, Desktop, and Terminal. The Git tab will show any changed files. In this example, we have not changed any files so it is empty. The Desktop tab will show the desktop environment. The terminal tab will show the terminal session.

Click on the Desktop tab. You will see a desktop that is the desktop of the virtual machine that the agent is running in. This can be very useful if the agent has been working on a visual update to your code. For example, an update to a dashboard or a report.

Click on the Terminal tab. This is a terminal session into the virtual machine that the agent is running in. Run the following commands to try and access the secret and environment variable.

```
echo $REPORT_EXPORT_KEY
```

```
echo $TEST_ENV_VAR
```

Key Takeaways & 2026 Best Practices

- Dockerfile + `.cursor/environment.json` = version-controlled, reproducible environments
- Secrets are attached manually when using Dockerfile-managed environments
- Use Cursor secrets only for low-risk or build-time values
- For real production runtime secrets → prefer external secret manager + MCP
- Agent-driven setup is convenient but less controllable than Dockerfile-managed

Debrief Questions (for class discussion)

1. What surprised you about the Dockerfile requirements?
2. Why is the "This repo" toggle important?
3. How would you apply this pattern to a real data/ML pipeline?